

We sincerely thank all reviewers for their constructive comments. We focus on the main concerns: QuantVGGT and FP baselines, deployment cost, TRE ablation, quantization granularity, calibration efficiency and ternary search efficiency. The final version will include all promised experiments, and we will open-source the code with complete implementation details.

Q1@AjH3,d4v7,nSXS: Why is QuantVGGT missing and where are the FP baselines? QuantVGGT was not open-sourced when we submitted this paper. Although its preprint and partial code were available, calibration set, and calibration functions were not released at that time, so a faithful matched comparison was not possible. We now reconstruct the setup and add a matched comparison under the same evaluation protocol and W4A8/W8A8 settings. We report dataset-level mean reconstruction accuracy (Acc.), which measures the average geometric reconstruction error; lower is better.

Setting	FP VGGT	TAPTQ (ours)	QuantVGGT
7Scenes W4A8	0.0176	0.0184	0.0189
7Scenes W8A8	0.0176	0.0175	0.0189
DTU W4A8	1.1849	1.158	1.187
DTU W8A8	1.1849	1.132	1.187
ETH3D W4A8	0.2933	0.281	0.276
ETH3D W8A8	0.2933	0.258	0.292

These results show that TAPTQ (ours) is competitive with QuantVGGT under matched settings. We also observe that quantized models can slightly outperform FP in several settings. A possible reason is that quantization acts as an implicit regularizer. Interval clipping suppresses noisy activation tails, and selective compensation recovers task-sensitive errors. We will report this as a setting-specific observation, not a universal claim. We will also add full per-scene and full-metric FP and QuantVGGT tables.

Q2@AjH3,wh68,d4v7: What is the deployment cost of compensation? The compensation branch is not a calibration-time artifact. During inference, the main backbone branch uses int8 quantization. The low-rank compensation branch remains in FP16 or bf16. Its rank and parameter count are small relative to the backbone. We measure VGGT-1B aggregator forward with 8 518²-pixel frames on B200, using 5 warmup runs and 25 timed runs.

Config.	Lat. (ms)	Speedup	Weight Mem.	Peak Mem.
FP32	401	1.00×	5.0GB	9.9GB
W8A8	74	5.42×	1.7GB	4.1GB
W8A16	81	4.95×	1.4GB	3.8GB

The measured compensation overhead is **0.041ms per module**. This is about 2.7% total overhead for the default 48 compensated modules, and 7.5% for a forced 144-module setting. W8A8 currently uses slightly more memory than W8A16 because our current implementation introduces extra dequantization and cache buffers. We will optimize this in the final implementation. We will also add matched FP16 VGGT, Pi3, and QuantVGGT timing with throughput.

Q3@wh68: Does TRE outperform simpler gating criteria? Yes. On 7Scenes W4A8, TRE gives the best Acc.-mean.

Method	25%	50%	Gap
TRE	0.0301	0.0264	0.0%
MSE	0.0306	0.0285	4.9%
Random	0.0393	0.0284	19.1%
Hessian	0.0344	0.0281	10.3%

- **TRE criterion:** TRE outperforms MSE, Random, and Hessian gating at both 25% and 50% keep ratios.
- **Selective compensation:** under $\tau = 0.007$, only 48 of 144 modules exceed the tail-relative-error threshold and are compensated. Unlike full-model residual compensation, TAPTQ (ours) targets only task-sensitive modules.
- **Additional baselines:** we will add top- k output-error gating. SVDQuant-style PTQ is relevant, but adapting its low-rank residual branch to VGGT and Pi3-style 3D architectures requires non-trivial engineering. We will include it in the final version.

Q4@wh68: How sensitive is TAPTQ (ours) to ρ , τ , and rank? The default values are $\rho = 0.01$, $\tau = 0.007$, and rank = 16. The τ ablation is already reported in Table 6. We add sweeps for ρ and rank on 7Scenes, VGGT W4A8.

ρ	.001	.01*	.05	.1	.5	1.0
Acc.	.036	.028	.029	.024	.029	.032
Rank	8	16*	32	64	128	256
Acc.	.028	.027	.030	.028	.028	.027

The sweep shows that TAPTQ (ours) is robust, not narrowly tuned. The default $\rho = 0.01$ already works well. A simple post-submission sweep finds that $\rho = 0.1$ further improves Acc. Rank is also stable. Rank 16 matches rank 256 with 16 $\times$ fewer parameters, which shows that a small low-rank correction is sufficient. Together with Table 6, these sweeps cover ρ , τ , and rank. The degradation at extreme settings also supports selective compensation. Compensating insensitive modules can add residual noise.

Q5@AjH3,d4v7: Why use tensor-wise results in the main tables? We agree this needs clarification. We use a unified per-tensor setting in the main tables to avoid mixing two factors: the PTQ method and the benefit of finer quantization granularity. Several baselines do not provide directly comparable channel-wise or group-wise versions. In our current channel-wise runs, TAPTQ (ours) still outperforms QuantVGGT and ERQ. Due to rebuttal space limits, we will include the full channel-wise table in the final version and supplement.

Q6@EHup: Why is calibration model-agnostic?

- **Model-agnostic:** as shown in Sec. 3.2, TAPTQ (ours) directly observes the data distribution for calibration-set construction. It does not rely on model-internal statistics or architecture-specific cues. In contrast, QuantVGGT uses VGGT-specific norm-statistics filtering and clustering. This must be re-derived for each new architecture. The VGGT and Pi3 results validate our model-agnostic design.
- **Encoder and stability:** we use a frozen pretrained encoder only once for calibration-subset construction. It is neither part of the quantized model nor used during inference. We will report its cost, mean and standard deviation over independent pools, and analyze discarded minority-cluster samples in the final version and supplement.

Q7@nSXS: How efficient is ternary search, and is it lossless? For interval search, we use the Hessian-based similarity metric in Eq. (3). At the same granularity, exhaustive grid search needs about 20,000 evaluations. Ternary search needs about 50 evaluations, giving roughly a **400 $\times$** reduction. Table 7 shows that ternary search is not completely lossless, but it saves much calibration time with only a small accuracy drop, which is mostly recovered by module-wise compensation. The similarity landscapes are almost always approximately unimodal. Representative cases are already reported, and more statistics will be added in the appendix.

Miscellaneous. Generality@d4v7. Our evaluation already covers VGGT and Pi3, two representative recent 3D geometry models. VGGT builds on the DUST3R and MAST3R line and is widely used in downstream applications. We will add DUST3R and MAST3R results in the final version. **Additional quantization settings@AjH3,d4v7.** We will add W4A4, FP camera-prediction, W6A6, and complete channel-wise tables in the final version and supplement. **Search baselines@nSXS.** Given the limited rebuttal window, we prioritize QuantVGGT, the most related 3D geometry quantization baseline. This also partially addresses the QuaRot concern, since QuantVGGT is an adapted and improved VGGT-specific extension over QuaRot-style rotation quantization. **Reproducibility and failure cases@d4v7,EHup.** We will release code, calibration details, preprocessing, and exact baseline configurations. We will add scene-level success and failure cases, analyze discarded minority-cluster samples, and fix all presentation issues.